Report on
Implementation of DES Using Python (CFB Mode)

Prepared by: Kimia Sadat Karbasi

Student ID: 60393958

Email: kimia.karbasi@ue-germany.de

Course/No: Software Optimization – P1SO01

Subject: Implementation of DES in CFB Mode

Professor Name: Dr. Rand Kouatly

Example.No: Task4

Introduction

The Data Encryption Standard (DES) is a symmetric-key block cipher, as it introduces fundamental concepts in block cipher design and modes of operation. This report presents a Python implementation of DES in Cipher Feedback (CFB) mode using the PyCryptodome cryptographic library.

Overview

This report provides a comprehensive analysis of a DES-CFB (Data Encryption Standard in Cipher Feedback mode) implementation and its integration with OpenSSL, presented through four systematic phases

1. Encryption Data: Initialization, Input Handling, Key Management, IV creation, Cipher Configuration, Encryption Execution, and Output Generation.
2. Decryption Data: Initialization, Input Handling, Cipher Reinitialization, Decryption Execution, and Output.
3. OpenSSL Implementation and Comparison with the DES Encryption.
4. Comparative Analysis

1. Encryption Data in CFB Mode

The following is a step-by-step analysis of the encryption workflow:

1.1 Initialization (PyCryptodome library)

In this step, necessary cryptographic modules are being imported such as the encryption algorithm, and get_random_bytes, which generates secure keys/IV

```
#KimiaSadatKarbasi-SID60393958
from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
import os
import binascii
```

Figure 1 – Importing Libraries

1.2 File Setup and Reading Input

File paths and encoded text are being defined in a binary format.

```
def main():
    plaintext_path = '/opt/DES/Task4/plain.txt'     # Input file to encrypt
    ciphertext_path = '/opt/DES/Task4/cipher.txt'   # Output encrypted file
    key_path = '/opt/DES/Task4/key.txt'            # File to store encryption keys
```

Figure 2 – Reading Input

```python
# Read plaintext (try both UTF-8 and binary fallback)
try:
    with open(plaintext_path, 'r', encoding='utf-8') as f:
        plaintext = f.read().encode('utf-8')
except UnicodeDecodeError:
    with open(plaintext_path, 'rb') as f:
        plaintext = f.read()
```

Figure 3 – Encoding Text in binary Format

1.3 Key/IV Generation

It creates a random 64-bit key and IV

```python
# Generate cryptographic material
key = get_random_bytes(8)  # DES uses 56-bit key + 8 parity bits
iv = get_random_bytes(8)   # Initialization Vector (block size)
```

Figure 4 – Key Generation

1.4 Cipher Setup

In this step, the configuration for DES in CFB mode will be set.

```python
# Initialize DES in CFB mode
# - key: Our generated encryption key
# - MODE_CFB: Cipher Feedback Mode selection
# - iv: Initialization Vector
# - segment_size=8: Process 8 bits (1 byte) at a time
    cipher = DES.new(key, DES.MODE_CFB, iv=iv, segment_size=8)
```

Figure 5 – Cipher Setup

1.5 Encryption

1.5.1 Encrypts the plain text (Encrypt IV with DES)

1.5.2 XOR result with plaintext

```python
# Perform encryption
 # The actual encryption happens here:
# - DES-CFB processes the plaintext in segments
# - Each segment is XORed with encrypted feedback
    ciphertext = cipher.encrypt(plaintext)
```

Figure 6 – Encryption of the plain Text

1.6 Output Handling

1.6.1 Convert binary to hex

1.6.2 Saving ciphertext and keys in the defined paths

```
# Convert ciphertext to UTF-8 hexadecimal
hex_ciphertext = binascii.hexlify(ciphertext).decode('utf-8')


with open(ciphertext_path, 'w', encoding='utf-8') as f:
    f.write(hex_ciphertext)

with open(key_path, 'w') as f:
    f.write(f"Key: {key.hex()}\n")
    f.write(f"IV: {iv.hex()}\n")
```

Figure 7 – Output Handing

1.7 Verification

In this step, encryption details will be printed.

```
    # Print results
    print("\nDES-CFB Encryption Results:")
    print(f"Key: {key.hex()}")
    print(f"IV: {iv.hex()}")
    print("\nCiphertext (UTF-Hex):")
    print(hex_ciphertext)


if __name__ == "__main__":
    print("This is Task4-CFB-enc.py")
    print("Kimia Sadat Karbasi - Student ID Number ='60393958'")
    main()
```

Figure 8 - Encryption Details

1.8 Results

Here are all the screenshots of the encryption code executed on a Linux machine. Before executing the Python code, we have to make sure that the library is installed:

sudo pip install pycryptodome

1.8.1 Display Output (Key, IV, and Ciphertext)

```
root@Docker-PAM:/opt/DES/Task4# ls
plain.txt  Task4-CFB-dec.py  Task4-CFB-enc.py  Task4-CFB-openssl.py
root@Docker-PAM:/opt/DES/Task4# python3 Task4-CFB-enc.py
This is Task4-CFB-enc.py
Kimia Sadat Karbasi - Student ID Number ='60393958'

DES-CFB Encryption Results:
Key: 9e6be112f8bfa4a7
IV: 5098f0ef3095eddb
```

Figure 9 – Output Key

```
Ciphertext (UTF-Hex):
718a61e33a6448b9142c36561e4a5d2979325caafa511330886f73cc55335cf6a670abc1c05c807ce04718622c
a92063c68a3091ff7a5856b30c97a202c518fc105099e5e4a97bd95c7463121ef2d8252722e3eefaa861a24102
b305b55a8e696b020d0001bd3fece335b4e2afeccd9695ec2f99b7d937509d03e5a581f2f087a1471f5e471875
2dea72e2c07a3470ed73725219671a355044442003b87f7bd3fc9ad4d4187999f91a7a3c23e8e4e9225128af7d
062c7b59cb4672d3429cb4c36046eab78acbb44b70db204c355bcb4022715d219603b84875e649e71aa171330a
1343358c920ec1239fe41fc55bd89f01896eaf4f826cf2ed800cb20c81cca10c242b658c0f602dce68e1e7a780
aba9922df272d6e4a37b41502d932b96f8ed79ea69e8dcbb1813f7b8725e936e85a1bad7d8d9883ba59a4447ef
b8c07c163c4f990ff5e6b5a50c90c413a9c26dce4acf50e12301e98be324fb68be5605f16a2ec3caf2c5138e61
218d5c960168fc053520c695fe1b048807b9c4ca5b0e69dd34f71fe2a5812a84b9714a4e7fdfd69b0244c6b9e5
51098afe852b4e82cfaa08355197701f3b722da4b70f96a5a056695b4876b3eb4359a11c7edf74d540844a60a8
f093637512bd9d5b485f1ed626d8665074af05be612be41ad0b46254d96243e6a05238e9c9457827fee2c8cf00
835d6bc9e61ac65819ceb23d6376fec415d0dda56fdfa4dc88a486ce49ff268fe1fbc62ce1d35f67eab984f9ac
13569dfd37f5660dc3c0e4b36d9ffd12d981f0ac77e0b1dbb3de883ca618ea71737a3cc344b3fa4d448db2d262
935568379c40a4d02a6da636b22c7b67db93b62504c41cd04bcfa069db626d4c486e960ea6426582016ed52f2e
edec73aa7cea25c30d5e9b3c6735d44e21f2329a99d47341e8017e85e01e721e2577b351412bfcdcb8efa92a2d
cd3214536b0e065d5787b2f3e2130520eb6dbe08d355595f1f40071f87dd924fc1bc62be1c037f1a166a903095
5b4469fd8ae3f2a54e389c15ec7a1c4412ba19370510ed3e22c17cd14a056682f1a07e71390081a6436e626263
67d133078ddcf5894f4e3ddcfd4e861e4974f63b6be8d63171e719f83753cea143a9d9db2dc697d91295077a90
736e7ca29735bfd24b45e60b09647bf244ae449652adacc069905904f980c07ab32359a0a6ab1b2ca6a18da310
86691bfa9e6fe85d0fd2f1866ef8d45cc41e3846e676d19eca65bf123baea2b2b9445b992ef869daf481d5b027
f7bb92a0153e378a5f5c909685b6657cdcbd4841f271f0d195e9f390be220f961e407a4184d958c6321123a94f
d5a2e4c6703708dde02d86b9ca563f16baea98fab94f35bde3835d58e097bf0d711154a2c34ebefb8706db5370
d422e6a57d9a7e785d5012414b8b4f3ac70dfb6df6af248e88142d5efcd407635e6b9e6106b2366f5d687899a2
a3757317f515e1603a1f56851cdf24dcd611178c0551d026977e8aaa8516c86fd93faddd712265b94bad5fe95f
96023a28502b82927dae5efbf1afbee59166a6a7284168e0c1e00ea4d2d46066ed99c59f751a4d7a0ae4fb034b
```

Figure 10 – Output Ciphertext

1.8.2 Showing Output from the created file (key.txt and cipher.txt)

```
root@Docker-PAM:/opt/DES/Task4# ls
cipher.txt  key.txt  plain.txt  Task4-CFB-dec.py  Task4-CFB-enc.py
root@Docker-PAM:/opt/DES/Task4# cat key.txt
Key: 9e6be112f8bfa4a7
IV: 5098f0ef3095eddb
root@Docker-PAM:/opt/DES/Task4#
```

Figure 11 – Key Text

```
root@Docker-PAM:/opt/DES/Task4# cat cipher.txt
718a61e33a6448b9142c36561e4a5d2979325caafa511330886f73cc55335cf6a670ab
a92063c68a3091ff7a5856b30c97a202c518fc105099e5e4a97bd95c7463121ef2d825
b305b55a8e696b020d0001bd3fece335b4e2afeccd9695ec2f99b7d937509d03e5a581
2dea72e2c07a3470ed73725219671a355044442003b87f7bd3fc9ad4d4187999f91a7a
062c7b59cb4672d3429cb4c36046eab78acbb44b70db204c355bcb4022715d219603b8
1343358c920ec1239fe41fc55bd89f01896eaf4f826cf2ed800cb20c81cca10c242b65
aba9922df272d6e4a37b41502d932b96f8ed79ea69e8dcbb1813f7b8725e936e85a1ba
b8c07c163c4f990ff5e6b5a50c90c413a9c26dce4acf50e12301e98be324fb68be5605
218d5c960168fc053520c695fe1b048807b9c4ca5b0e69dd34f71fe2a5812a84b9714a
51098afe852b4e82cfaa08355197701f3b722da4b70f96a5a056695b4876b3eb4359a1
f093637512bd9d5b485f1ed626d8665074af05be612be41ad0b46254d96243e6a05238
835d6bc9e61ac65819ceb23d6376fec415d0dda56fdfa4dc88a486ce49ff268fe1fbc6
13569dfd37f5660dc3c0e4b36d9ffd12d981f0ac77e0b1dbb3de883ca618ea71737a3c
935568379c40a4d02a6da636b22c7b67db93b62504c41cd04bcfa069db626d4c486e96
edec73aa7cea25c30d5e9b3c6735d44e21f2329a99d47341e8017e85e01e721e2577b3
cd3214536b0e065d5787b2f3e2130520eb6dbe08d355595f1f40071f87dd924fc1bc62
5b4469fd8ae3f2a54e389c15ec7a1c4412ba19370510ed3e22c17cd14a056682f1a07e
67d133078ddcf5894f4e3ddcfd4e861e4974f63b6be8d63171e719f83753cea143a9d9
736e7ca29735bfd24b45e60b09647bf244ae449652adacc069905904f980c07ab32359
86691bfa9e6fe85d0fd2f1866ef8d45cc41e3846e676d19eca65bf123baea2b2b9445b
f7bb92a0153e378a5f5c909685b6657cdcbd4841f271f0d195e9f390be220f961e407a
d5a2e4c6703708dde02d86b9ca563f16baea98fab94f35bde3835d58e097bf0d711154
d422e6a57d9a7e785d5012414b8b4f3ac70dfb6df6af248e88142d5efcd407635e6b9e
a3757317f515e1603a1f56851cdf24dcd611178c0551d026977e8aaa8516c86fd93fad
96023a28502b82927dae5efbf1afbee59166a6a7284168e0c1e00ea4d2d46066ed99c5
```

Figure 12 – Cat Cipher Text

This is the part of code that helps to read ciphertext and converts the hex to binary. Details are shown as commands in the code below:

```python
# Read ciphertext (Hex to binary Conversion)
try:
    with open(ciphertext_path, 'r') as f:  # Open in text mode (not 'rb')
        hex_ciphertext = f.read().strip()  # Read hex string
        ciphertext = bytes.fromhex(hex_ciphertext)  # Convert to binary

except Exception as e:
    print(f"Error reading ciphertext: {e}")
    return
```

Figure 13 – Reading Ciphertext / Converting Hex to binary

2. Decryption Data in CFB Mode

The decryption process is explained step by step in the following section.

2.1 Initialization (PyCryptodome Library)

```python
#KimiaSadatKarbasi-SID60393958
from Crypto.Cipher import DES
import binascii
```

Figure 14 – Importing Library

2.2 File Setup and Reading Inputs

The key, IV, and ciphertext are read and converted to binary.

```python
def main():
    # File paths
    ciphertext_path = '/opt/DES/Task4/cipher.txt'
    key_path = '/opt/DES/Task4/key.txt'
    decrypted_path = '/opt/DES/Task4/decrypted.txt'

    # Read key and IV (Hex to Binary Conversion)
    try:
        with open(key_path, 'r') as f:
            key = bytes.fromhex(f.readline().split(': ')[1].strip())
            iv = bytes.fromhex(f.readline().split(': ')[1].strip())
    except Exception as e:
        print(f"Error reading key file: {e}")
        return
```

Figure 15 – Reading Inputs

2.3 Cipher Setup

```python
# Initialize DES-CFB cipher
cipher = DES.new(key, DES.MODE_CFB, iv=iv, segment_size=8)
```

Figure 16 – Cipher Setup

2.4 Decryption

2.4.1 Encrypts the ciphertext (decrypt IV with DES)

2.4.2 XOR result with ciphertext

```python
decrypted = cipher.decrypt(ciphertext)
```

Figure 17 – Decrypting Cipher Text

2.5 Output Handling

2.5.1 Convert binary to hex
2.5.2 Saving ciphertext, keys in the defined paths, and showing them.

```python
# Write decrypted output
try:
    with open(decrypted_path, 'wb') as f:
        f.write(decrypted)
    print(f"\nSuccessfully decrypted to: {decrypted_path}")

    # Try to print as text (UTF-8) or hex if binary

    print("\nDecrypted text content:")
    print(decrypted.decode('utf-8'))

except Exception as e:
    print(f"Error writing decrypted file: {e}")
```

Figure 18 – Python Code for Output Handling

2.6 Results

Here are all the screenshots of the decryption code executed on a Linux machine.

2.6.1 Display the decrypted text during the Python code

```
root@Docker-PAM:/opt/DES/Task4# ls
cipher.txt  decrypted.txt  key.txt  plain.txt  Task4-CFB-dec.py  Ta
root@Docker-PAM:/opt/DES/Task4# cat decrypted.txt
"The Clockmaker of Rothenburg"
In the heart of Bavaria, nestled between rolling green hills and
cobblestone streets, sat the town of Rothenburg ob der Tauber.
Known for its perfectly preserved medieval walls and timber-framed
houses, the town looked like a postcard from the past. But what tru
gave Rothenburg its charm was the little clock shop on
Schmiedgasse Street, run by the old clockmaker, Herr Baumann.
Herr Baumann had lived in Rothenburg all his life. His small shop,
Zeit & Seele—Time and Soul—was filled with all kinds of clocks:
cuckoo clocks with dancing figurines, grandfathers with deep chimes
and delicate pocket watches that glimmered like stardust. People
came from all over Germany—and even farther—to see his creations.
But what they didn't know was that Herr Baumann's clocks held more
than gears and springs. They held memories.
It was said that when Herr Baumann built a clock, he carved into it
tiny piece of someone's story. A couple in love, a soldier coming
home, a child's laughter—he somehow captured these moments, and
his clocks ticked with life, echoing not just time but feeling.
One chilly April morning, a young boy named Lukas wandered into
the shop. He was ten, wearing a red scarf too big for his neck, and
had curious eyes that never stopped moving.
"Guten Tag," said Herr Baumann with a smile. "Looking for
something?"
Lukas shook his head. "Just looking. Papa says you make magic
clocks."
```

Figure 19 - Decrypted text during the Python

2.6.2 Output show (decrypted.txt)

```
root@Docker-PAM:/opt/DES/Task4# python3 Task4-CFB-dec.py
This is Task4-CFB-dec.py
Kimia Sadat Karbasi - Student ID Number ='60393958'

Successfully decrypted to: /opt/DES/Task4/decrypted.txt

Decrypted text content:
"The Clockmaker of Rothenburg"
In the heart of Bavaria, nestled between rolling green hills and
cobblestone streets, sat the town of Rothenburg ob der Tauber.
Known for its perfectly preserved medieval walls and timber-framed
houses, the town looked like a postcard from the past. But what truly
gave Rothenburg its charm was the little clock shop on
Schmiedgasse Street, run by the old clockmaker, Herr Baumann.
Herr Baumann had lived in Rothenburg all his life. His small shop,
Zeit & Seele—Time and Soul—was filled with all kinds of clocks:
cuckoo clocks with dancing figurines, grandfathers with deep chimes,
and delicate pocket watches that glimmered like stardust. People
came from all over Germany—and even farther—to see his creations.
But what they didn't know was that Herr Baumann's clocks held more
than gears and springs. They held memories.
It was said that when Herr Baumann built a clock, he carved into it a
tiny piece of someone's story. A couple in love, a soldier coming
home, a child's laughter—he somehow captured these moments, and
his clocks ticked with life, echoing not just time but feeling.
One chilly April morning, a young boy named Lukas wandered into
the shop. He was ten, wearing a red scarf too big for his neck, and
had curious eyes that never stopped moving.
"Guten Tag," said Herr Baumann with a smile. "Looking for
something?"
Lukas shook his head. "Just looking. Papa says you make magic
clocks."
```

Figure 20 – Decrypted Text output

3. Encryption Data with OpenSSL

The Encryption process with OpenSSL is explained step by step in the following section:

3.1 Initialization

Necessary modules are being imported, such as subprocess, used for the OpenSSL command in the Linux machine.

```
#KimiaSadatKarbasi-SID60393958
#Using this for the Linux command
import subprocess
```

Figure 21- Importing Module

3.2 Encryption with OpenSSL

3.2.1 File Setup and Reading Input

```
# File paths
PLAINTEXT_PATH = '/opt/DES/Task4/plain.txt'
KEY_PATH = '/opt/DES/Task4/key.txt'
CIPHERTEXT_PATH = '/opt/DES/Task4/cipher.txt'
CIPHERTEXT_OPENSSL_PATH = '/opt/DES/Task4/cipher-openssl.txt'
DECRYPTED_OPENSSL_PATH = '/opt/DES/Task4/decrypted-openssl.txt'

def openssl_des_cfb():
    #Read Key and IV from the text
    with open(KEY_PATH, 'r') as f:
        key_hex = f.readline().split(': ')[1].strip()
        iv_hex = f.readline().split(': ')[1].strip()

    #Read plaintext in binary mode
    with open(PLAINTEXT_PATH, 'rb') as f:
        plaintext = f.read()
```

Figure 22 - Reading Inputs and File Setups

3.2.2 Preparing OpenSSL Command and Executing Encryption

Parameters include CFB mode, Key hex, IV Hex, and legacy provider, with high version 3.3 for supporting CFB mode will be used for OpenSSL Command.

```
# Encrypt with OpenSSL (DES-CFB)
openssl_cmd = [
    'openssl', 'enc', '-des-cfb',  #CFB mode
    '-K', key_hex,
    '-iv', iv_hex,

    #For using CFB mode in Openssl we need legacy provider
    '-provider', 'legacy'
]

result = subprocess.run(
    openssl_cmd,
    input=plaintext,
    capture_output=True,
    check=True
)
```

Figure 23 – OpenSSL Command

```python
# Save decrypted output
with open(DECRYPTED_OPENSSL_PATH, 'wb') as f:
    f.write(result_decrypt.stdout)
```

Figure 24 – Decrypted Output

3.3 Decrypt with OpenSSL

3.3.1 Reading Ciphertext

```python
# Read DES ciphertext (hex format)
with open(CIPHERTEXT_PATH, 'r') as f:
    des_ciphertext_hex = f.read().strip()
```

Figure 25 – Reading Ciphertext

```python
# Decrypt with OpenSSL (verify)
openssl_decrypt_cmd = [
    'openssl', 'enc', '-d', '-des-cfb8',
    '-K', key_hex,
    '-iv', iv_hex,
    '-provider', 'legacy'
]
```

Figure 26 – Decryption with OpenSSL

```python
# Convert hex ciphertext to binary for decryption
ciphertext_bin = bytes.fromhex(ciphertext_hex)
result_decrypt = subprocess.run(
    openssl_decrypt_cmd,
    input=ciphertext_bin,
    capture_output=True,
    check=True
)
```

Figure 27 – Converting Cipher Text to Binary

3.5 Output Handling

Display results and verify correctness (Ciphertext match, and Decrypted matches the original)

```
    # Compare hex ciphertexts
    print("\nOpenSSL (DES-CFB) Results")
    print(f"Key: {key_hex}")
    print(f"IV: {iv_hex}")
    print(f"OpenSSL Ciphertext (first 32 chars): {ciphertext_hex[:32]}")
    print(f"DES Ciphertext (first 32 chars): {des_ciphertext_hex[:32]}")
    print(f"Ciphertexts match: {ciphertext_hex == des_ciphertext_hex}")
    print(f"\nDecrypted matches original: {result_decrypt.stdout == plaintext}")

if __name__ == "__main__":
    print("This is Task4-CFB-openssl.py")
    print("Kimia Sadat Karbasi - Student ID Number ='60393958'")
    openssl_des_cfb()
```

Figure 28 – Displaying Results

3.6 Results

3.6.1 Displaying Details

Details such as key, IV, OpenSSL Cipher text, DES Ciphertext, Matching Ciphertexts, and matching OpenSSL will be decrypted with the original text.

```
root@Docker-PAM:/opt/DES/Task4# python3 Task4-CFB-openssl.py
This is Task4-CFB-openssl.py
Kimia Sadat Karbasi - Student ID Number ='60393958'

OpenSSL (DES-CFB) Results
Key: 9e6be112f8bfa4a7
IV: 5098f0ef3095eddb
OpenSSL Ciphertext (first 32 chars): 718a61e33a6448b9142c36561e4a5d29
DES Ciphertext (first 32 chars): 718a61e33a6448b9142c36561e4a5d29
Ciphertexts match: True

Decrypted matches original: True
root@Docker-PAM:/opt/DES/Task4#
```

Figure 29 – Results

```
root@Docker-PAM:/opt/DES/Task4# cat decrypted-openssl.txt
"The Clockmaker of Rothenburg"
In the heart of Bavaria, nestled between rolling green hills and
cobblestone streets, sat the town of Rothenburg ob der Tauber.
Known for its perfectly preserved medieval walls and timber-framed
houses, the town looked like a postcard from the past. But what truly
gave Rothenburg its charm was the little clock shop on
Schmiedgasse Street, run by the old clockmaker, Herr Baumann.
Herr Baumann had lived in Rothenburg all his life. His small shop,
Zeit & Seele—Time and Soul—was filled with all kinds of clocks:
cuckoo clocks with dancing figurines, grandfathers with deep chimes,
and delicate pocket watches that glimmered like stardust. People
came from all over Germany—and even farther—to see his creations.
But what they didn't know was that Herr Baumann's clocks held more
than gears and springs. They held memories.
It was said that when Herr Baumann built a clock, he carved into it a
tiny piece of someone's story. A couple in love, a soldier coming
home, a child's laughter—he somehow captured these moments, and
his clocks ticked with life, echoing not just time but feeling.
One chilly April morning, a young boy named Lukas wandered into
the shop. He was ten, wearing a red scarf too big for his neck, and
had curious eyes that never stopped moving.
"Guten Tag," said Herr Baumann with a smile. "Looking for
something?"
Lukas shook his head. "Just looking. Papa says you make magic
clocks."
Herr Baumann chuckled, kneeling down so his eyes met Lukas's.
"Magic? Well, maybe a little."
The boy's smile faded. "Do you make clocks that can go backward?"
The clockmaker blinked. "Backward?"
"My mama used to bring me here," Lukas whispered. "Before she got
sick. I want to go back… to a time when she was okay."
Silence filled the shop like snow falling quietly. Herr Baumann stood
and gently placed a hand on Lukas's shoulder. "Come with me."
```

Figure 30 – Decrypted Openssl Text

```
root@Docker-PAM:/opt/DES/Task4# cat cipher
cipher-openssl.txt  cipher.txt
root@Docker-PAM:/opt/DES/Task4# cat cipher-openssl.txt
718a61e33a6448b9142c36561e4a5d2979325caafa511330886f73cc55335cf6a670abc1c05c807ce04718622cdc
a92063c68a3091ff7a5856b30c97a202c518fc105099e5e4a97bd95c7463121ef2d8252722e3eefaa861a2410279
b305b55a8e696b020d0001bd3fece335b4e2afeccd9695ec2f99b7d937509d03e5a581f2f087a1471f5e471875cc
2dea72e2c07a3470ed73725219671a355044442003b87f7bd3fc9ad4d4187999f91a7a3c23e8e4e9225128af7d21
062c7b59cb4672d3429cb4c36046eab78acbb44b70db204c355bcb4022715d219603b84875e649e71aa171330a6d
1343358c920ec1239fe41fc55bd89f01896eaf4f826cf2ed800cb20c81cca10c242b658c0f602dce68e1e7a780e4
aba9922df272d6e4a37b41502d932b96f8ed79ea69e8dcbb1813f7b8725e936e85a1bad7d8d9883ba59a4447efce
b8c07c163c4f990ff5e6b5a50c90c413a9c26dce4acf50e12301e98be324fb68be5605f16a2ec3caf2c5138e6176
218d5c960168fc053520c695fe1b048807b9c4ca5b0e69dd34f71fe2a5812a84b9714a4e7fdfd69b0244c6b9e555
51098afe852b4e82cfaa08355197701f3b722da4b70f96a5a056695b4876b3eb4359a11c7edf74d540844a60a84d
f093637512bd9d5b485f1ed626d8665074af05be612be41ad0b46254d96243e6a05238e9c9457827fee2c8cf00d5
835d6bc9e61ac65819ceb23d6376fec415d0dda56fdfa4dc88a486ce49ff268fe1fbc62ce1d35f67eab984f9ac0e
13569dfd37f5660dc3c0e4b36d9ffd12d981f0ac77e0b1dbb3de883ca618ea71737a3cc344b3fa4d448db2d2625f
935568379c40a4d02a6da636b22c7b67db93b62504c41cd04bcfa069db626d4c486e960ea6426582016ed52f2e9f
edec73aa7cea25c30d5e9b3c6735d44e21f2329a99d47341e8017e85e01e721e2577b351412bfcdcb8efa92a2d93
cd3214536b0e065d5787b2f3e2130520eb6dbe08d355595f1f40071f87dd924fc1bc62be1c037f1a166a903095a1
5b4469fd8ae3f2a54e389c15ec7a1c4412ba19370510ed3e22c17cd14a056682f1a07e71390081a6436e626263ec
67d133078ddcf5894f4e3ddcfd4e861e4974f63b6be8d63171e719f83753cea143a9d9db2dc697d91295077a9015
736e7ca29735bfd24b45e60b09647bf244ae449652adacc069905904f980c07ab32359a0a6ab1b2ca6a18da310c3
86691bfa9e6fe85d0fd2f1866ef8d45cc41e3846e676d19eca65bf123baea2b2b9445b992ef869daf481d5b0270c
f7bb92a0153e378a5f5c909685b6657cdcbd4841f271f0d195e9f390be220f961e407a4184d958c6321123a94fa2
d5a2e4c6703708dde02d86b9ca563f16baea98fab94f35bde3835d58e097bf0d711154a2c34ebefb8706db53704b
d422e6a57d9a7e785d5012414b8b4f3ac70dfb6df6af248e88142d5efcd407635e6b9e6106b2366f5d687899a293
```

Figure 31 – Cipher Openssl Text

4. Comparative analysis

In this phase, we present the comparison between DES-CFB implementation using the PyCryptodome library and OpenSSL's DES-CFB, including a detailed table and explanations.

Now we compare encryption/decryption workflows, Implementation, performance, security, and practical usage of both implementations.

4.1 Encryption/Decryption Workflow Comparison

Step	Custom DES-CFB	OpenSSL DES-CFB
Key Setup	Read from key.txt (manual parsing)	Read from key.txt (auto parsed)
Encryption	Auto-handled using library	Auto-handled using openssl command
Padding	Manual	Automatic
Output	Save Ciphertext as binary	Save Ciphertext as binary
Decryption	Reverse Process	Single Command (openssl enc)
Error Handling	Manual checks	Built-in

4.2 Implementation & Design

Aspect	Custom DES-CFB	OpenSSL DES-CFB
Codebase	Handwritten (Python)	Precompiled OpenSSL binary
Mode Support	Manual CFB logic	Built-in -des-cfb flag
Providers	-	Requires -provider legacy (v3.0+)

4.3 Performance & Optimization

Aspect	Custom DES-CFB	OpenSSL DES-CFB
Speed	Slower(Python interpreter)	Faster(optimized)
Mode Support	Manual CFB logic	Built-in -des-cfb flag
Hardware acceleration	No	Uses CPU

4.4 Security

Aspect	Custom DES-CFB	OpenSSL DES-CFB
Side-Channel	Vulnerable (lack of constant-time)	Hardened against timing attacks
Padding	Manual handling	Automatic
Maintenance	Your responsibility	Regularly updated

4.5 Practical Use

Aspect	Custom DES-CFB	OpenSSL-CFB
Debugging	Complex	Simple
Portability	Depends on Code	Works everywhere
Command	Using pycryptodome library	openssl enc -des-cfb8
Mode	Testing/Learning	Production

Conclusion

This report shows how to implement DES-CFB encryption by going through four structured phases: data encryption, decryption, OpenSSL integration, and comparative analysis. It was demonstrated during the comparison that OpenSSL's performance is improved by the use of hardware acceleration and security features. While our customized implementation was a valuable educational tool for understanding, OpenSSL proved that it can be more efficient and reliable for practical applications.